DAL-585
AY 2026-27 Autumn Semester
Programming Assignment 1

Due Date

Friday 28 August 2026 at 5pm. Late assignments will not be accepted and will receive ZERO mark.

Objective

In this assignment, you are to write a program (in Python v3) to perform part- of-speech (POS) tagging. Given a sentence, the task is to assign a POS tag to each word in the sentence. We will adopt the Penn Treebank tag set in this assignment.

For example, given the following input sentence:

```
He also is a consensus manager .
```

The POS tagger should output the following:

```
He/PRP also/RB is/VBZ a/DT consensus/NN
manager/NN ./.
```

Note that each input sentence has been tokenized (punctuation symbols are separated from words).

You are to implement a POS bigram tagger based on a hidden Markov model, in which the probability of the POS tag of the current word is conditioned on the POS tag of the previous word. The Viterbi algorithm will be used to find the optimal sequence of most probable POS tags.

Training and Testing

You are provided with a training set of POS-tagged sentences (`train.tags`). You will train your POS tagger using this training set.

The command for training (and if necessary tuning) the POS tagger is:

```
Python3 build_tagger.py train.tags model_file
```

The file `model_file` contains the statistics gathered from the training (and if necessary tuning) process, which include the POS tag transition probabilities and the word emission probabilities (and other tuned parameters if necessary).

A separate development set of sentences is also provided (`dev.sents`) along with its POS tags (dev.tags). You use these two files to measure the performance of your POS tagger.

The command to test and generate an output file is:

```
Python3 run_tagger.py dev.sents model_file
dev.out
```

The output file dev.out has the same format as the POS-tagged training file. A sample output file is also provided (sample_test`.out`).

To measure the performance of your tagger, you can use the MeasurePOSTaggerAccuracy.py file. The command for this is as follows:

Python3 MeasurePOSTaggerAccuracy.py dev.out dev.tags

First argument of this command must be your output file and second argument must be the gold-label POS tag file.

The commands `build_tagger.py` and `run_tagger.py` as shown above will be executed to evaluate your POS tagger, except that testing will be done on a set of new, blind test sentences. The blind test file consists of a list of sentences (without POS tags), one sentence per line. A sample test file is provided (sample_`test.sents`).

Do not use hard coded file paths in your code. Files such as model_file and sent.out file must be generated within CWD.

Ideally execution time of build_tagger.py and run_tagger.py should not cross 5 min each.

Deliverables

You are to work on this programming assignment *individually*. You must hand in the following:
1. Your source code (with documentation if needed). Note that graphical user interface is not needed.
2. A short report in PDF format (no more than 4 pages) describing details of your implemented method. It must include the running time of the build_tagger.py and run_tagger.py. It must include the performance of your POS tagger on the dev.sents file. The report must

be prepared on overleaf.com using ACL format. TA will help you with this.

Email your program and report to TA Shyamal Das (shyamal_d@mfs.iitr.ac.in) by the due date. Your program and report should be emailed as one single attached zip file, with the email subject line clearly indicating course ID, your name (as it appears on your student ID card), and your roll number. An example email subject line is:

DAL585 Arnab_Nandi 1234567

Grading

50% for your report, which includes the method used to build your POS tagger (how unknown words are handled, smoothing method used, etc).
50% for the accuracy of your POS tagger, as measured on the blind test set of sentences.